МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №4.2
по дисциплине
«Программирование»

Выполнила студентка группы ИВТб-1304-06-00 _____/Забелина А.А.

Проверил ст. преподаватель кафедры ЭВМ _____/Крутиков А.К.

Киров 2026

Цель

Цель работы: научиться создавать простые графические интерфейсы с использованием библиотеки tkinter, организовывать обработку событий и интегрировать ранее разработанные программные модули в единое приложение. Закрепить понимание работы выбранной структуры данных через её визуальное представление.

Задание

На основе реализации структуры данных из лабораторной работы 4 необходимо разработать оконное приложение с графическим интерфейсом на Python.
Требования:

1. Все операции со структурой данных должны выполняться исключительно через вызовы функций отдельно оформленных библиотечных модулей.

2. Необходимо предусмотреть реализацию, как минимум двух модулей: один на C++, второй - Python.

3. Необходима реализация двух вариантов C++ модуля: с использованием структур на динамической памяти и с использованием контейнеров и адаптеров STL.

4. Интерфейс должен адекватно реагировать на попытку выполнить некорректную операцию (например, Pop из пустого стека), выводя понятное сообщение, без аварийного завершения программы.

5. Визуализация должна обновляться после каждой успешной операции.

6. В качестве ответа необходимо прикрепить отчёт и ссылку на репозиторий с проектом.

Теория

Циклический двусвязный список - это динамическая структура данных, состоящая из узлов, каждый из которых содержит минимум три основных поля: информационное поле, указатель на следующий элемент, указатель на предыдущий элемент.

Наличие двух указателей позволяет перемещаться по списку в обоих направлениях.

Цикличность заключается в том, что указатель на предыдущий элемент первого элемента ссылается на последний, а последний элемент указывает не на nil, а на первый элемент.

Структура циклического двусвязного списка:

В стеке хранится указатель на «начало» списка, а также его длина, для более эффективной работы со списком. В куче хранятся сами элементы (узлы).

Структура одного узла:

inf и *ch* - два информационных поля, хранят число и символ соответственно;

prev - указатель на предыдущий элемент;

next - указатель на следующий элемент.

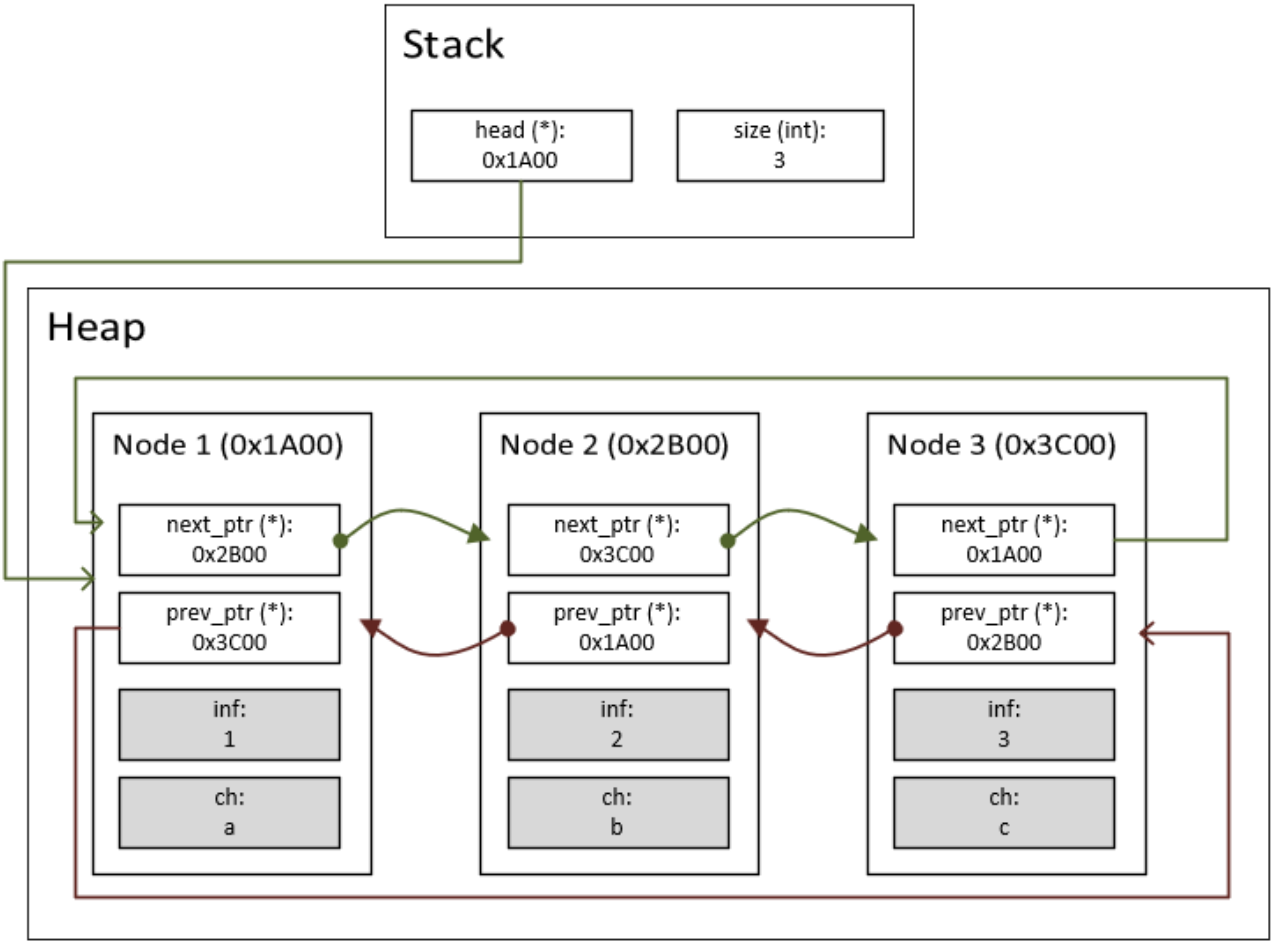

Рисунок 1 — Циклический двусвязный список в памяти

Схемы программы

Схемы алгоритмов взаимодействия с циклическим двусвязным списком являются едиными для всех четырёх реализаций (C, C++ STL, Pascal, Python). Это обусловлено тем, что все библиотеки реализуют одинаковый набор операций с идентичной логикой работы: вставка элемента, удаление элемента, поиск по значению и по символу, получение длины и очистка списка.

Функция Insert позволяет вставлять элементы в список: в условный конец списка, после указанного элемента или перед указанным элементом, в зависимости от переданного параметра.

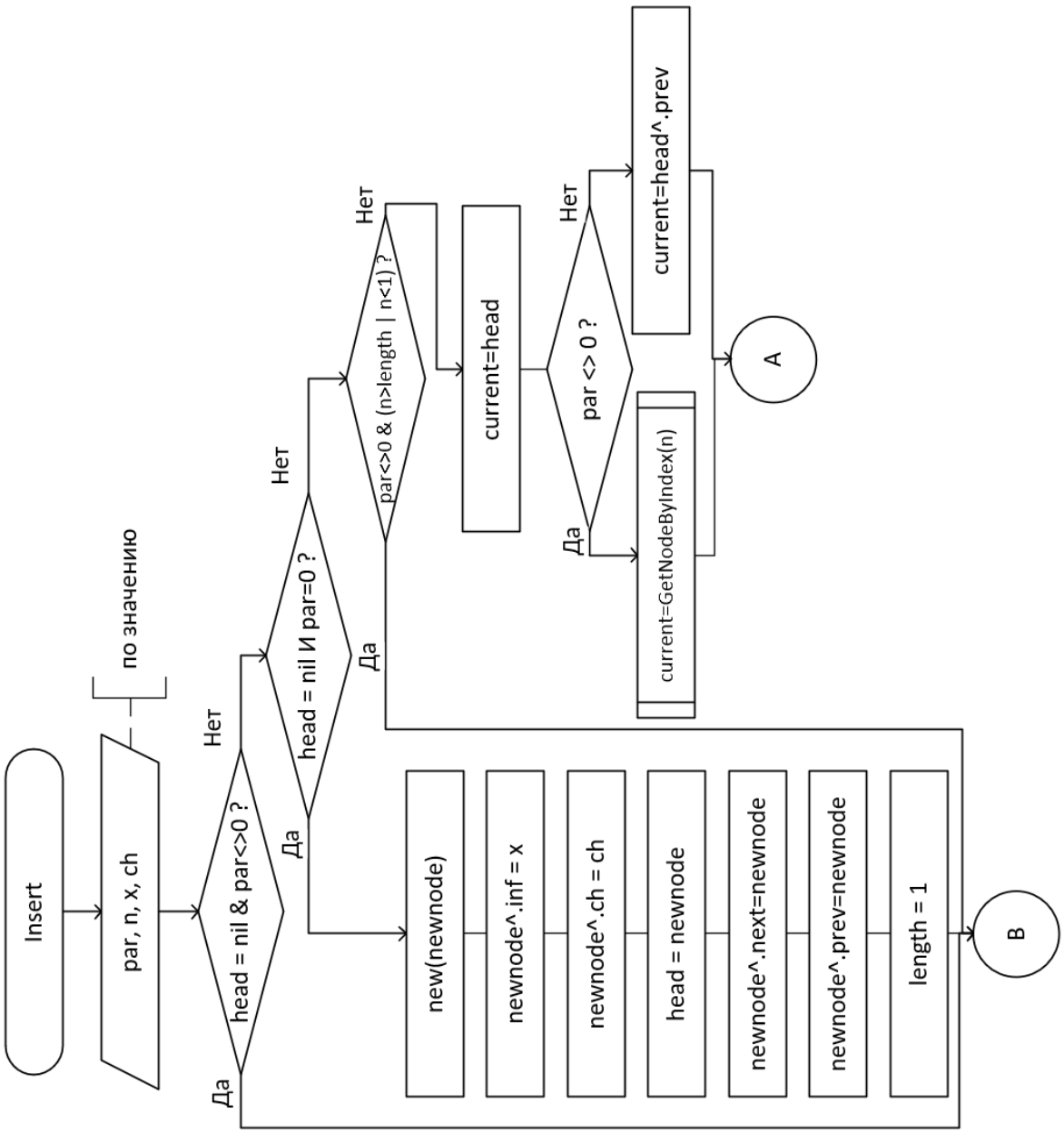

Рисунок 2 — Функция Insert

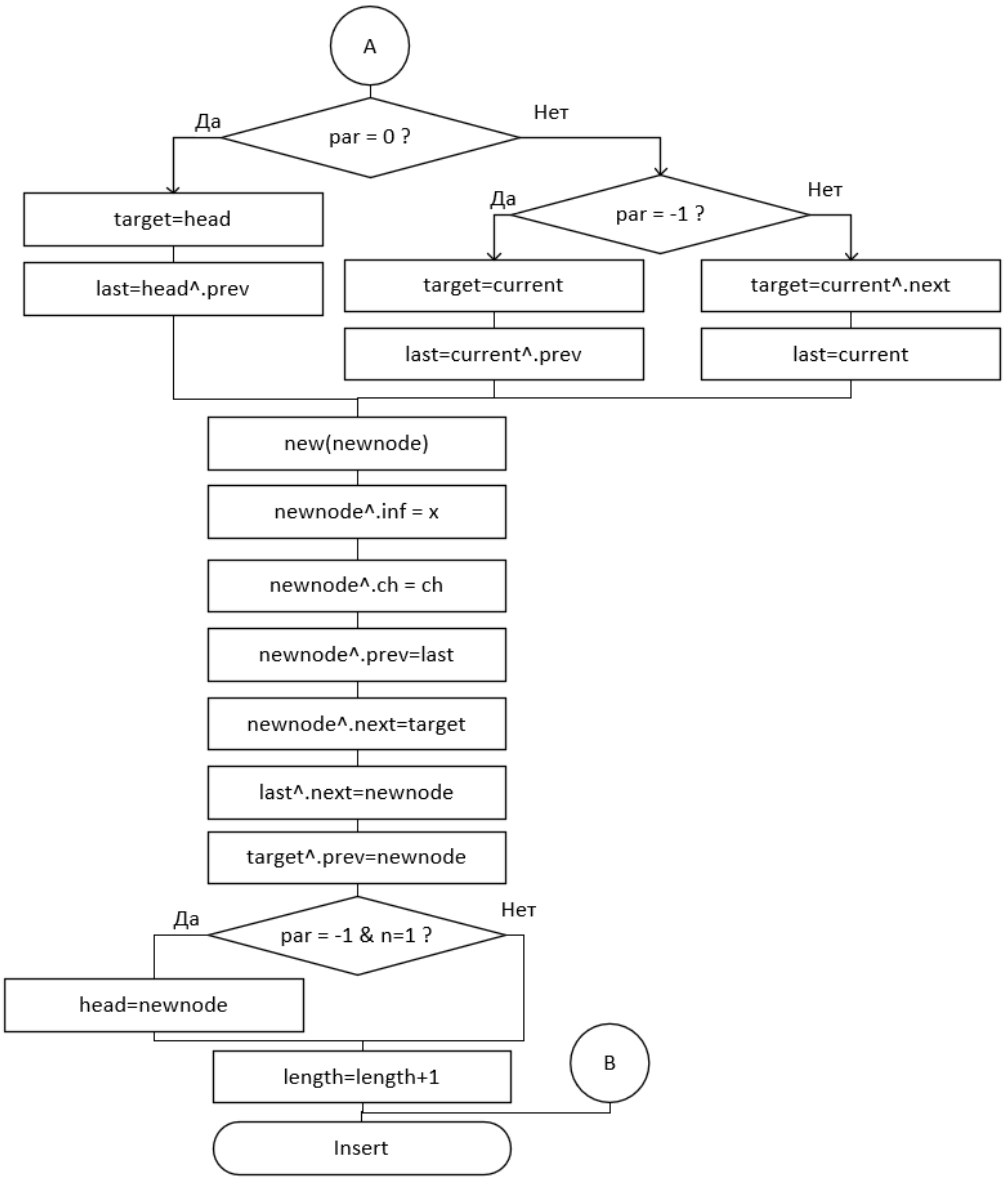

Рисунок 3 — Функция Insert (продолжение)

Функция Remove используется для удаления элементов. Можно удалить выбранный элемент, элемент перед или после указанного, в зависимости от указанного параметра.

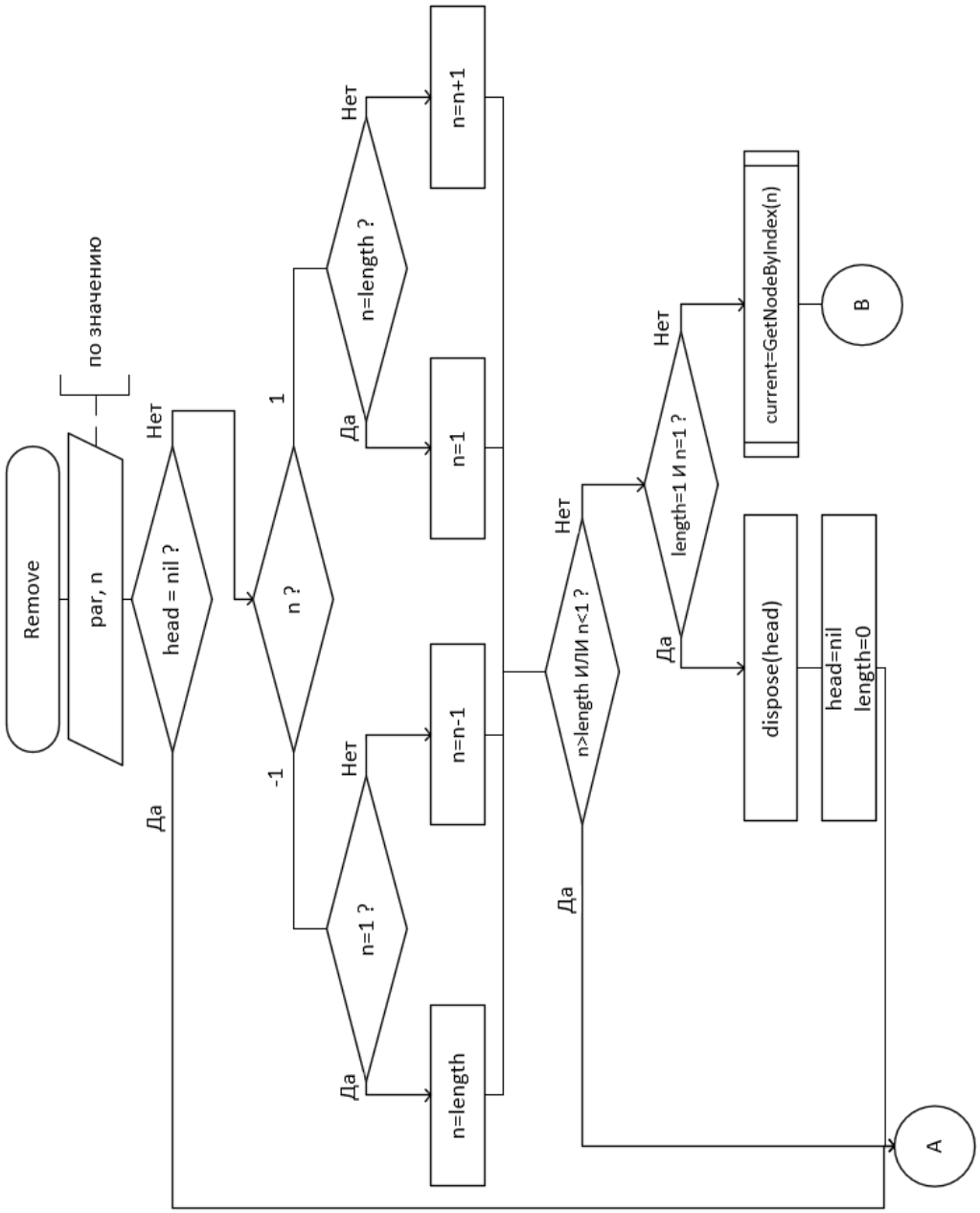

Рисунок 4 — Функция Remove

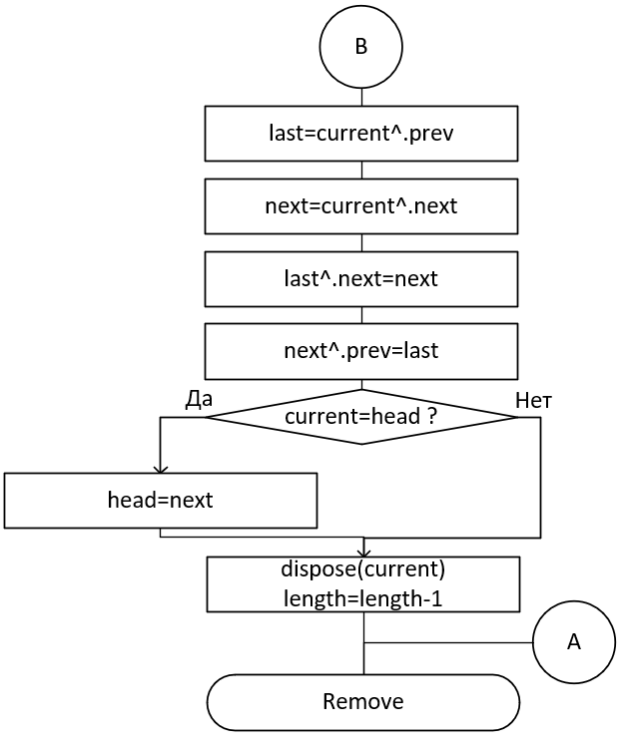

Рисунок 5 — Функция Remove (продолжение)

Функция Clear позволяет полностью очистить список.

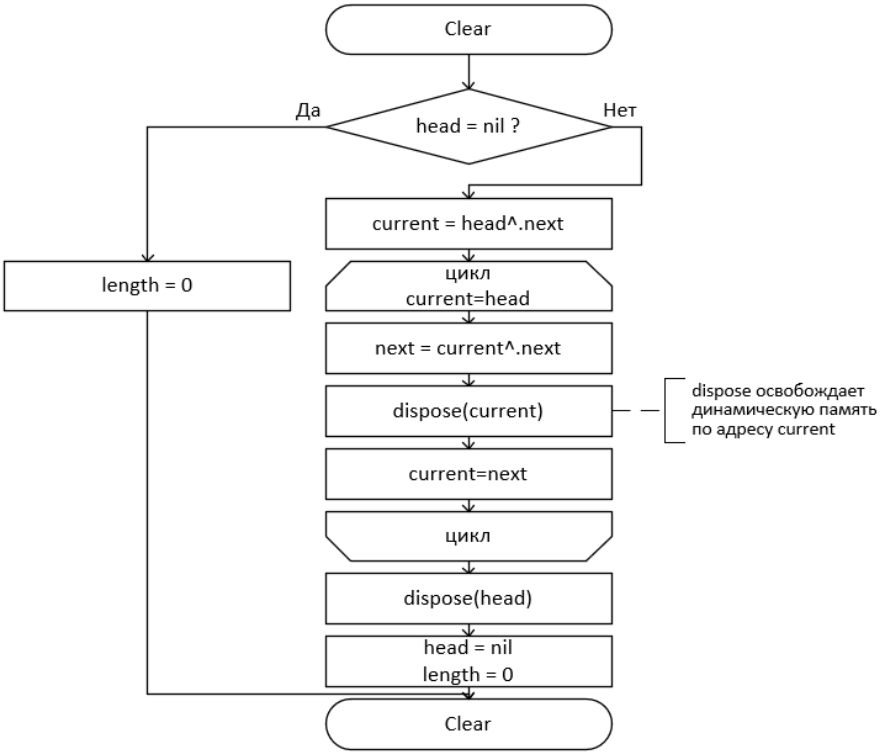

Рисунок 6 — Функция Clear

Функция GetNodeByIndex является вспомогательной и позволяет получить узел списка по заданному индексу.

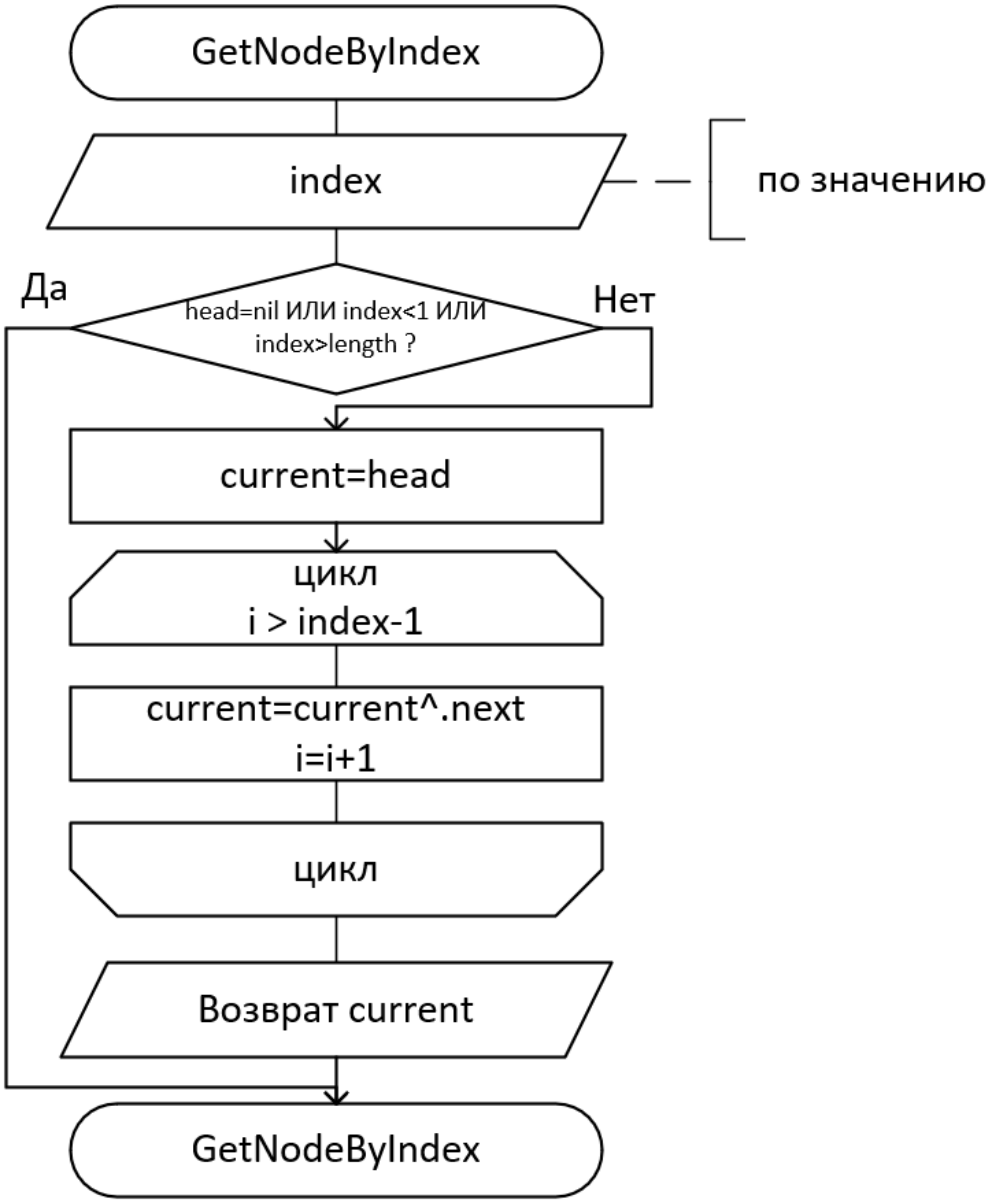

Рисунок 7 — Функция GetNodeByIndex

По этим схемам алгоритмов был написан код для каждой библиотеки. Поскольку логика работы одинакова во всех реализациях, в отчёте представлен один набор схем алгоритмов.

Структурная схема отображает взаимодействие частей программы между собой. Динамические библиотеки подключаются к Python-приложению при помощи стандартной библиотеки ctypes, что обеспечивает простую настройку и взаимодействие, а также совместимость. В каждой библиотеке при помощи процедур и функций реализовано управление структурой динамической памяти - циклическим двусвязным списком.

Для языков C и Pascal используется явное управление памятью (`malloc/New` и `free/Dispose`).

Для языка C++ используются STL-контейнеры, а именно `std::list`. Цикличность списка обеспечивается при помощи математического расчёта (остаток от деления). Следующий элемент рассчитывается как:

$$next = (index + 1)\%length$$

а предыдущий как:

$$prev = (index - 1 + length)\%length$$

где $index$ - текущий элемент списка, $length$ - длина списка. Прибавление $length$ перед взятием остатка необходимо для избежания отрицательных значений при вычислении предыдущего индекса.

На Python нет возможности использовать указатели напрямую, поэтому для работы со списком используются классы со структурой, аналогичной остальным библиотекам. Подключение python-файла к основному приложению осуществляется при помощи `import`.

Основное Python-приложение состоит из описания интерфейса при помощи Tkinter, определения методов для корректной работы библиотек (указание argtypes, restype, CDLL для взаимодействия с динамическими библиотеками), а также обработчиков событий для связывания интерфейса с динамической библиотекой циклического двусвязного списка.

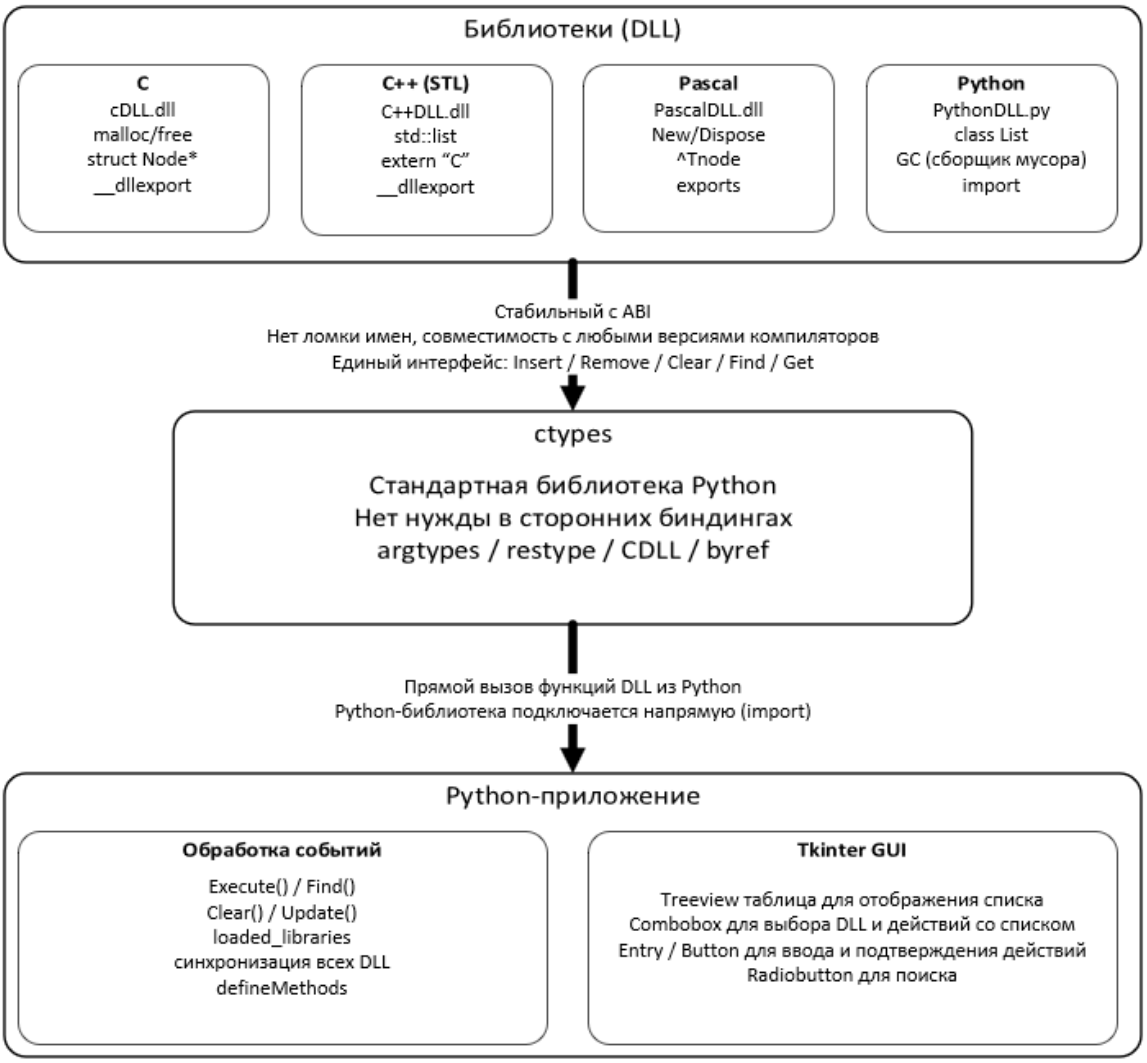

Рисунок 8 — Структурная схема приложения

Экранные формы

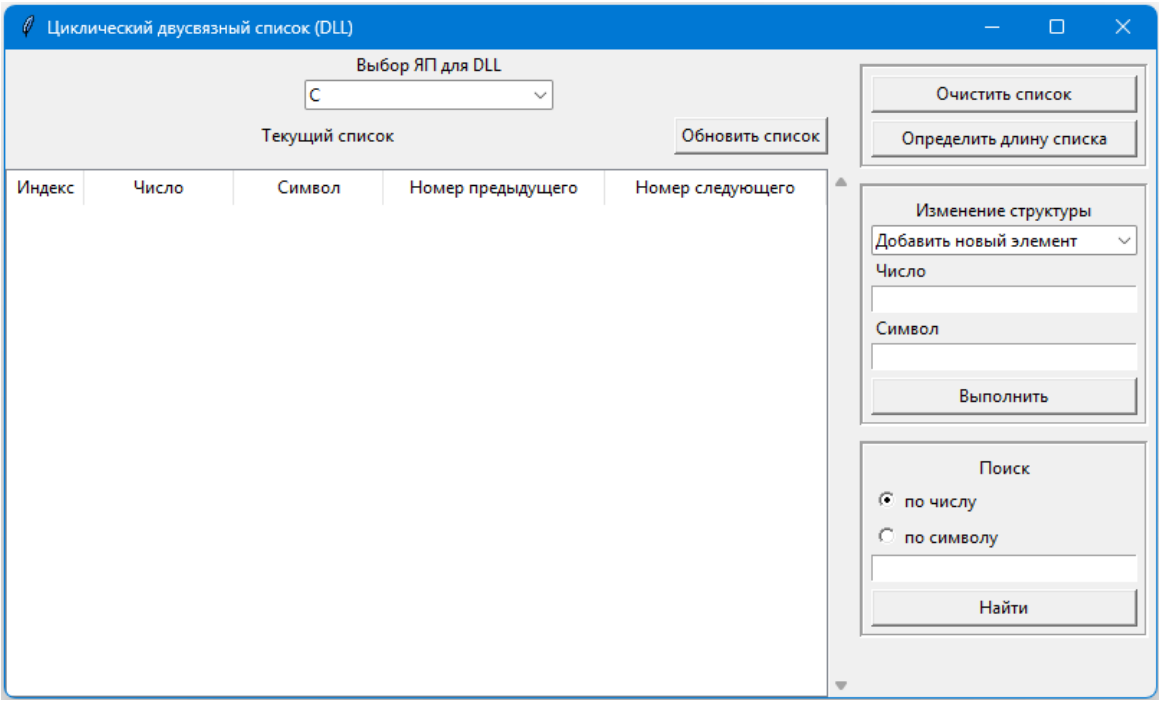

Рисунок 9 — Основная форма приложения

Выбор языка программирования для подключаемой библиотеки осуществляется через выпадающий список в верхней части интерфейса. Переключение между библиотеками незаметно для пользователя - данные во всех библиотеках синхронизированы.

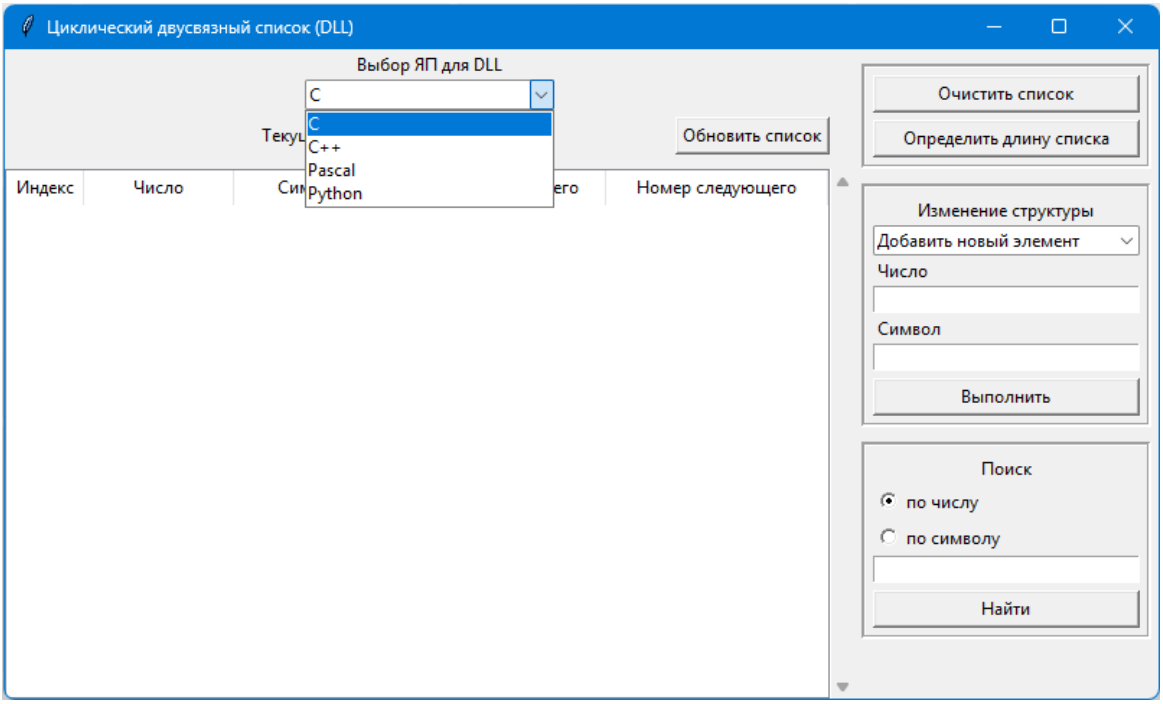

Рисунок 10 — Выбор языка программирования

Выбор действия осуществляется через выпадающий список. В зависимости от выбранного действия отображаются соответствующие поля ввода.

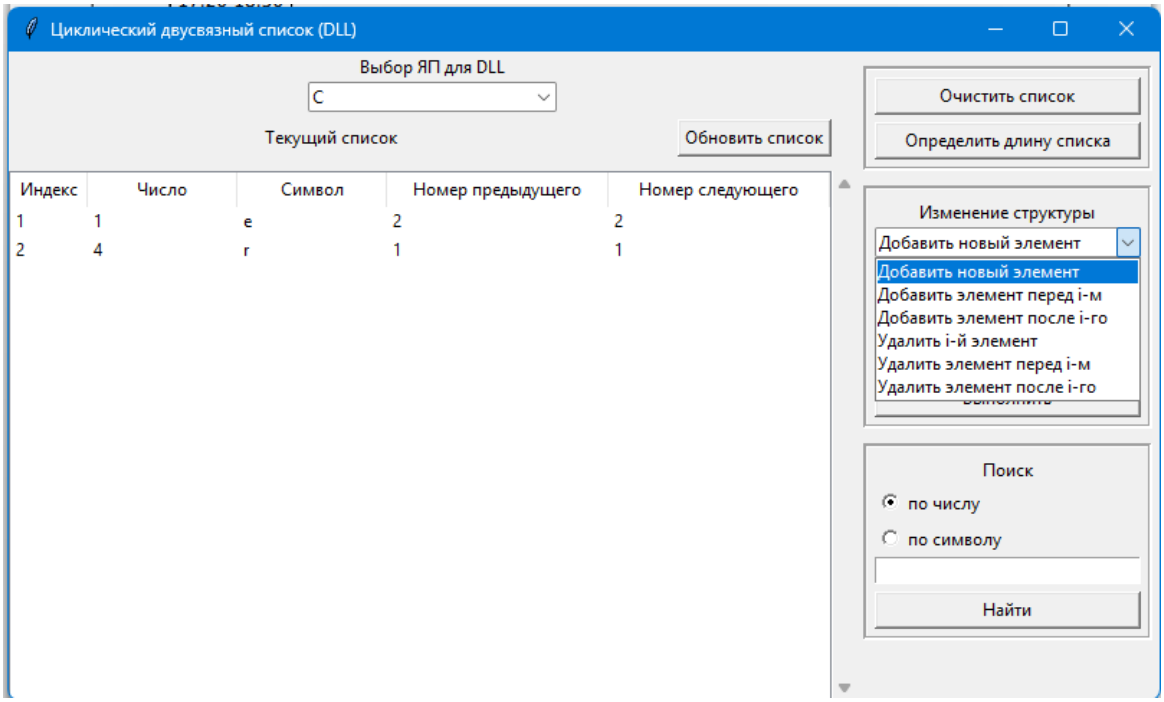

Рисунок 11 — Выбор действия

Поиск элементов осуществляется по числовому полю или по символу. Результат отображается в виде сообщения с индексами найденных элементов.

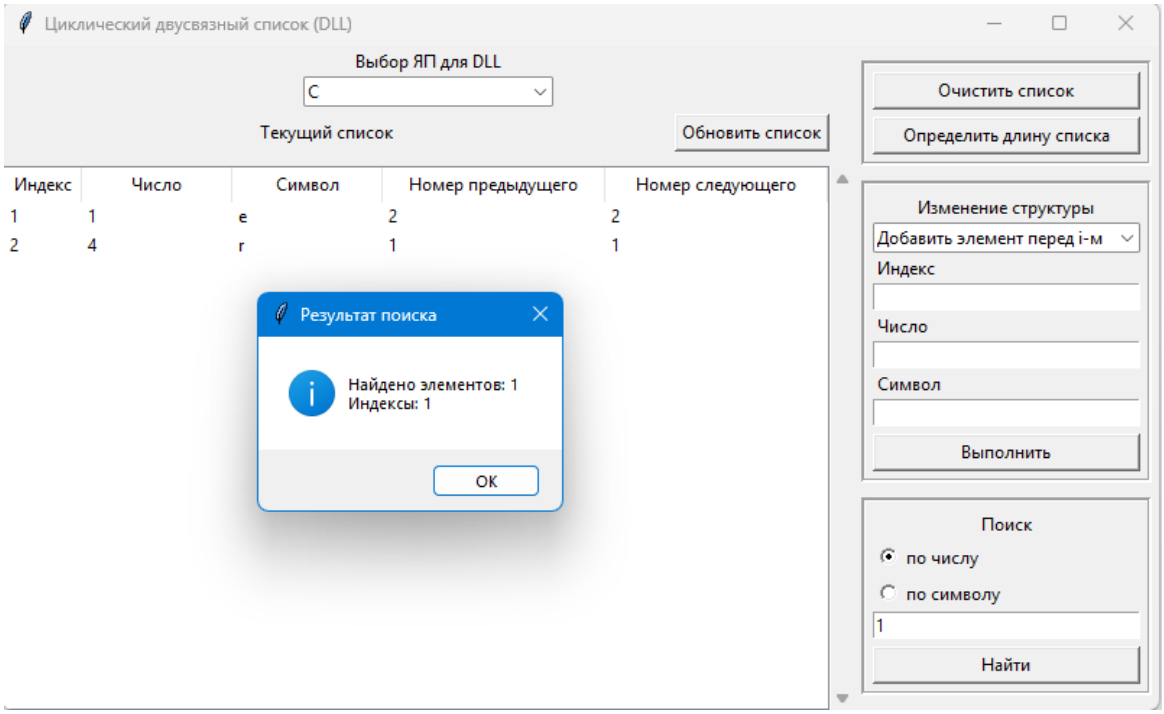

Рисунок 12 — Поиск элемента

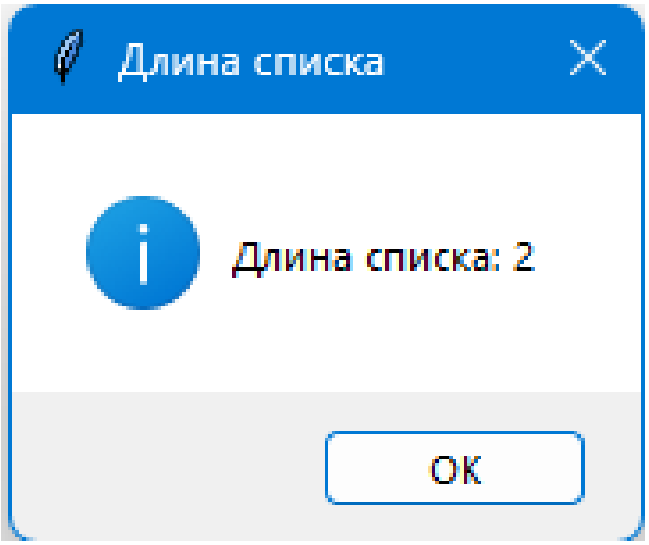

Рисунок 13 — Определение длины списка

Исходный код

Pascal библиотека

```
library PascalDLL;
uses
  System.SysUtils,
  System.Classes;
type
  PList=^TList;
  TList=record
    inf:integer;
    ch:char;
    prev:PList;
    next:PList;
  end;
type
  TIntegerArray = array[0..65535] of Integer;
  PIntegerArray = ^TIntegerArray;
var length:integer=0;
head:PList=nil;
function GetNodeByIndex(index:integer):PList;
var current:PList;
i:integer;
begin
  Result:=nil;
  if (head=nil) or (index<1) or (index>length) then
      Exit;
  current:=head;
  for i:=1 to index-1 do
    current:=current^.next;
  Result:=current;
end;
function Insert(par,n,x:integer;ch:char):integer; cdecl;
var i:integer;
current,target,last,newnode:PList;
begin
Result:=0;
if (head=nil) and (par<>0) then
begin
  Result:=1;
  Exit;
end;
if (head=nil) and (par=0) then
begin
  New(newnode);
  newnode^.inf:=x;
  newnode^.ch:=ch;
  head:=newnode;
  newnode^.next:=newnode;
  newnode^.prev:=newnode;
  length:=1;
  Exit;
end;
if (par<>0) and ((n>length) or (n<1)) then
begin
  Result:=1;
  Exit;
end;
if par<>0 then
  current:=GetNodeByIndex(n)
else
  current:=head^.prev;
```

```
if par=0 then
begin
  target:=head;
  last:=head^.prev;
end
else if par=-1 then
begin
  target:=current;
  last:=current^.prev;
end
else
begin
  target:=current^.next;
  last:=current;
end;
New(newnode);
newnode^.inf:=x;
newnode^.ch:=ch;
newnode^.prev:=last;
newnode^.next:=target;
last^.next:=newnode;
target^.prev:=newnode;
if (par=-1) and (n=1) then head:=newnode;
length:=length+1;
end;
function Remove(par,n:integer):integer; cdecl;
var current,last,next:PList;
begin
Result:=0;
if head=nil then begin Result:=1; Exit; end;
case par of
-1: if n=1 then n:=length else n:=n-1;
 1: if n=length then n:=1 else n:=n+1;
end;
if (n>length) or (n<1) then begin Result:=1; Exit; end;
if (length=1) and (n=1) then
begin
dispose(head);
head:=nil;
length:=0;
Exit;
end;
current:=GetNodeByIndex(n);
last:=current^.prev;
next:=current^.next;
last^.next:=next;
next^.prev:=last;
if current=head then head:=next;
dispose(current);
length:=length-1;
end;
procedure Clear(); cdecl;
var current,next:PList;
begin
if head=nil then
begin
  length:=0;
  exit;
end;
current:=head^.next;
while current<>head do
begin
  next:=current^.next;
  dispose(current);
  current:=next;
end;
```

```
dispose(head);
head:=nil;
length:=0;
end;
function GetLength():integer; cdecl;
begin
  Result:=length;
end;
function FindByInf(x:integer; out count:integer):PInteger; cdecl;
var i:integer;
current:PList;
resultArray:PIntegerArray;
begin
  count:=0;
  Result:=nil;
  if head=nil then Exit;
  current:=head;
  for i:=1 to length do
    begin
      if current^.inf=x then
        count:=count+1;
      current:=current^.next;
    end;
    if count=0 then Exit;
    GetMem(resultArray, count*SizeOf(integer));
    current:=head;
    count:=0;
    for i:=1 to length do
      begin
        if x=current^.inf then
        begin
          resultArray^[count]:=i;
          count:=count+1;
        end;
        current:=current^.next;
      end;
    Result:=PInteger(resultArray);
end;
function FindByChar(x:integer; out count:integer):PInteger; cdecl;
var i:integer;
current:PList;
resultArray:PIntegerArray;
begin
  count:=0;
  Result:=nil;
  if head=nil then Exit;
  current:=head;
  for i:=1 to length do
    begin
      if Ord(current^.ch)=x then
        count:=count+1;
      current:=current^.next;
    end;
    if count=0 then Exit;
    GetMem(resultArray, count*SizeOf(integer));
    current:=head;
    count:=0;
    for i:=1 to length do
      begin
        if x=Ord(current^.ch) then
        begin
          resultArray^[count]:=i;
          count:=count+1;
        end;
        current:=current^.next;
      end;
```

```
    Result:=PInteger(resultArray);
end;
procedure FreeArray(p:PInteger);cdecl;
begin
  if p<>nil then
    FreeMem(p);
end;
function GetInf(index:integer):integer;cdecl;
var node:PList;
begin
  Result:=0;
  node:=GetNodeByIndex(index+1);
  if node <> nil then
  Result:=node^.inf;
end;
function GetChar(index:integer):integer;cdecl;
var node:PList;
begin
  Result:=0;
  node:=GetNodeByIndex(index+1);
  if node <> nil then
  Result:=Ord(node^.ch);
end;
function GetNext(index:integer):integer;cdecl;
var node,target:PList;
i:integer;
begin
  Result:=0;
  node:=GetNodeByIndex(index+1);
  target:=node^.next;
  node:=head;
  for i:= 1 to length do
    begin
      if node=target then
        begin Result:=i-1; Exit; end;
      node:=node^.next;
    end;
end;
function GetPrev(index:integer):integer;cdecl;
var node,target:PList;
i:integer;
begin
  Result:=0;
  node:=GetNodeByIndex(index+1);
  target:=node^.prev;
  node:=head;
  for i:= 1 to length do
    begin
      if node=target then
        begin Result:=i-1; Exit; end;
      node:=node^.next;
    end;
end;
exports
  Insert,
  Remove,
  Clear,
  GetLength,
  FindByInf,
  FindByChar,
  FreeArray,
  GetInf,
  GetChar,
  GetNext,
  GetPrev;
begin
```

```
end.
```

Заголовочный файл C библиотеки (List.h)

```
#pragma once
typedef struct Node Node;
__declspec(dllexport) int Insert(int par, int n, int x, char ch);
__declspec(dllexport) int Remove(int par, int n);
__declspec(dllexport) void Clear(void);
__declspec(dllexport) int GetLength(void);
__declspec(dllexport) int* FindByInf(int x, int* count);
__declspec(dllexport) int* FindByChar(int x, int* count);
__declspec(dllexport) void FreeArray(int* p);
__declspec(dllexport) int GetInf(int index);
__declspec(dllexport) int GetChar(int index);
__declspec(dllexport) int GetNext(int index);
__declspec(dllexport) int GetPrev(int index);
```

C библиотека (List.c)

```
#include "List.h"
#include <stdlib.h>
struct Node {
        int inf;
        char ch;
        struct Node* prev;
        struct Node* next;};
static Node* head = NULL;
static int length = 0;
static Node* GetNodeByIndex(int index) {
        if (head == NULL || index < 1 || index > length) {
                return NULL;}
        Node* current;
        current = head;
        for (int i = 1;i < index;i++) {
                current = current->next;}
        return current;}
int Insert(int par, int n, int x, char ch) {
        Node* newnode, * current, * target, * last;
        if (head == NULL && par != 0) {
                return 1;}
        if (head == NULL && par == 0) {
                newnode = (Node*)malloc(sizeof(Node));
                if (newnode == NULL) return 1;
                newnode->inf = x;
                newnode->ch = ch;
                head = newnode;
                newnode->next = newnode;
                newnode->prev = newnode;
                length = 1;
                return 0;}
        if (par != 0 && (n > length || n < 1)) {return 1;}
        if (par != 0) {current = GetNodeByIndex(n);}
        else {current = head->prev;}
    if (par == 0) {
                target = head;
                last = head->prev;}
        else {
```

```
                if (par == -1) {
                        target = current;
                        last = current->prev;}
                else {
                        target = current->next;
                        last = current;}}
        newnode = (Node*)malloc(sizeof(Node));
        if (newnode == NULL) return 1;
        newnode->inf = x;
        newnode->ch = ch;
        newnode->prev = last;
        newnode->next = target;
        last->next = newnode;
        target->prev = newnode;
        if (par == -1 && n == 1) {head = newnode;}
        length++;
        return 0;}

int Remove(int par, int n) {
        if (head == NULL) return 1;
        switch (par) {
        case -1: if (n == 1) n = length; else n -= 1; break;
        case 1: if (n == length) n = 1; else n += 1; break;}
        if (n > length || n < 1) return 1;
        if (length == 1 && n == 1) {
                free(head);
                head = NULL;
                length = 0;
                return 0;}
        Node* current = GetNodeByIndex(n);
        Node* last = current->prev;
        Node* next = current->next;
        last->next = next;
        next->prev = last;
        if (current == head) head = next;
        free(current);
        length -= 1;
        return 0;}
void Clear() {
        if (head == NULL) {
                length = 0;
                return;}
        Node* current = head->next;
        while (current != head) {
                Node* next = current->next;
                free(current);
                current = next;}
        free(head);
        head = NULL;
        length = 0;}
int GetLength() {return length;}
int* FindByInf(int x, int* count) {
        *count = 0;
        if (head == NULL) return NULL;
        Node* current = head;
        for (int i = 1;i <= length;i++) {
                if (current->inf == x) (*count)++;
                current = current->next;}
        if (*count == 0) return NULL;
        int* resultArray = (int*)malloc((*count) * sizeof(int));
        current = head;
        *count = 0;
        for (int i = 1;i <= length;i++) {
                if (x == current->inf) {
```

```
                        resultArray[*count] = i;
                        (*count)++;}
                current = current->next;}
        return resultArray;}
int* FindByChar(int x, int* count) {
        *count = 0;
        if (head == NULL) return NULL;
        Node* current = head;
        for (int i = 1;i <= length;i++) {
                if (current->ch == x) (*count)++;
                current = current->next;}
        if (*count == 0) return NULL;
        int* resultArray = (int*)malloc((*count) * sizeof(int));
        current = head;
        *count = 0;
        for (int i = 1;i <= length;i++) {
                if (x == current->ch) {
                        resultArray[*count] = i;
                        (*count)++;}
                current = current->next;}
        return resultArray;}
void FreeArray(int* p) {
        if (p != NULL) free(p);}
int GetInf(int index) {
        Node* node = GetNodeByIndex(index + 1);
        if (node != NULL) return node->inf;
        return 0;}
int GetChar(int index) {
        Node* node = GetNodeByIndex(index + 1);
        if (node != NULL) return node->ch;
        return 0;}
int GetNext(int index) {
        Node* node = GetNodeByIndex(index + 1);
        Node* target = node->next;
        node = head;
        for (int i = 1;i <= length;i++) {
                if (node == target) return i - 1;
                node = node->next;}
        return 0;}
int GetPrev(int index) {
        Node* node = GetNodeByIndex(index + 1);
        Node* target = node->prev;
        node = head;
        for (int i = 1;i <= length;i++) {
                if (node == target) return i - 1;
                node = node->next;}
        return 0;}
```

C++ STL библиотека

```
#include <windows.h>
#include <list>
#include <vector>
#include <cstdlib>
struct Node {
    int inf;
    int ch;};
static std::list<Node> gList;
static std::list<Node>::iterator GetIterByIndex(int index) {
    if (index < 1 || index >(int)gList.size())
        return gList.end();
```

```
    auto it = gList.begin();
    std::advance(it, index - 1);
    return it;}
extern "C" {
    __declspec(dllexport) int Insert(int par, int n, int x, int ch) {
        Node newNode{ x, ch };
        if (gList.empty()) {
            if (par != 0) return 1;
            gList.push_back(newNode);
            return 0;}
        if (par == 0) {
            gList.push_back(newNode);
            return 0;}
        if (n < 1 || n >(int)gList.size()) return 1;
        auto it = GetIterByIndex(n);
        if (it == gList.end()) return 1;
        if (par == -1) {
            gList.insert(it, newNode);}
        else {
            ++it;
            gList.insert(it, newNode);}
        return 0;}
    __declspec(dllexport) int Remove(int par, int n) {
        if (gList.empty()) return 1;
        int size = (int)gList.size();
        int target = n;
        if (par == -1) {
            target = (n == 1) ? size : n - 1;}
        else if (par == 1) {
            target = (n == size) ? 1 : n + 1;}
        if (target < 1 || target > size) return 1;
        auto it = GetIterByIndex(target);
        if (it == gList.end()) return 1;
        gList.erase(it);
        return 0;}
    __declspec(dllexport) void Clear() {
        gList.clear();}
    __declspec(dllexport) int GetLength() {
        return (int)gList.size();}
    __declspec(dllexport) int* FindByInf(int x, int* count) {
        std::vector<int> found;
        int i = 1;
        for (auto& node : gList) {
            if (node.inf == x)
                found.push_back(i);
            ++i;}
        *count = (int)found.size();
        if (found.empty()) return nullptr;
        int* arr = (int*)malloc(found.size() * sizeof(int));
        for (int j = 0; j < (int)found.size(); ++j)
            arr[j] = found[j];
        return arr;}
    __declspec(dllexport) int* FindByChar(int ch, int* count) {
        std::vector<int> found;
        int i = 1;
        for (auto& node : gList) {
            if (node.ch == ch)
                found.push_back(i);
            ++i;}
        *count = (int)found.size();
        if (found.empty()) return nullptr;
        int* arr = (int*)malloc(found.size() * sizeof(int));
        for (int j = 0; j < (int)found.size(); ++j)
```

```
            arr[j] = found[j];
        return arr;}
    __declspec(dllexport) void FreeArray(int* ptr) {
        if (ptr) free(ptr);}
    __declspec(dllexport) int GetInf(int index) {
        auto it = GetIterByIndex(index + 1);
        if (it == gList.end()) return 0;
        return it->inf;}
    __declspec(dllexport) int GetChar(int index) {
        auto it = GetIterByIndex(index + 1);
        if (it == gList.end()) return 0;
        return it->ch;}
    __declspec(dllexport) int GetNext(int index) {
        int size = (int)gList.size();
        if (size == 0) return 0;
        return (index + 1) % size;}
    __declspec(dllexport) int GetPrev(int index) {
        int size = (int)gList.size();
        if (size == 0) return 0;
        return (index - 1 + size) % size;}}
```

Python библиотека

```
class Node:
    def __init__(self,inf,ch):
        self.inf=inf
        self.ch=ch
        self.prev=None
        self.next=None
class List:
    def __init__(self):
        self.head=None
        self.length=0
    def GetNodeByIndex(self,index):
        if self.head is None or index < 1 or index > self.length:
            return None
        current=self.head
        for _ in range(1,index):
            current=current.next
        return current
    def Insert(self,par,n,x,ch):
        if self.head is None and par!=0:
            return 1
        if self.head is None and par==0:
            newnode=Node(x,ch)
            newnode.next=newnode
            newnode.prev=newnode
            self.head=newnode
            self.length=1
            return 0
        if par!=0 and (n>self.length or n<1):
            return 1
        if par!=0:
            current=self.GetNodeByIndex(n)
        else:
            current=self.head.prev
        if par==0:
            target=self.head
            last=self.head.prev
        else:
            if par==-1:
                target=current
```

```
                last=current.prev
            else:
                target=current.next
                last=current
        newnode=Node(x,ch)
        newnode.prev=last
        newnode.next=target
        last.next=newnode
        target.prev=newnode
        if par==-1 and n==1:
            self.head=newnode
        self.length+=1
        return 0
    def Remove(self,par,n):
        if self.head is None: return 1
        if par==-1:
            if n==1: n=self.length
            else: n-=1
        elif par==1:
            if n==self.length: n=1
            else: n+=1
        if n>self.length or n<1: return 1
        if self.length==1 and n==1:
            self.head=None
            self.length=0
            return 0
        current=self.GetNodeByIndex(n)
        last=current.prev
        next=current.next
        last.next=next
        next.prev=last
        if current==self.head: self.head=next
        self.length-=1
        return 0
    def Clear(self):
        if self.head is None:
            self.length = 0
            return
        current = self.head.next
        while current != self.head:
            next_node = current.next
            current.prev = None
            current.next = None
            current = next_node
        self.head.prev = None
        self.head.next = None
        self.head = None
        self.length = 0
    def GetLength(self):
        return self.length
    def FindByInf(self,x):
        indices=[]
        if self.head is None: return indices
        current=self.head
        for i in range(1, self.length+1):
            if current.inf==x:
                indices.append(i)
            current=current.next
        return indices
    def FindByChar(self,ch):
        indices=[]
        if self.head is None: return indices
        current=self.head
        for i in range(1, self.length+1):
            if current.ch==ord(ch):
```

```
                indices.append(i)
            current=current.next
        return indices
    def GetInf(self,index):
        node=self.GetNodeByIndex(index+1)
        if node is None: return 0
        return node.inf
    def GetChar(self,index):
        node=self.GetNodeByIndex(index+1)
        if node is None: return 0
        return node.ch
    def GetNext(self,index):
        node=self.GetNodeByIndex(index+1)
        if node is None: return 0
        target=node.next
        node=self.head
        for i in range(1,self.length+1):
            if node==target: return i-1
            node=node.next
        return 0
    def GetPrev(self,index):
        node = self.GetNodeByIndex(index + 1)
        if node is None: return 0
        target = node.prev
        node = self.head
        for i in range(1, self.length + 1):
            if node == target: return i - 1
            node = node.next
        return 0
    def FreeArray(self, ptr):
        pass
```

Основное приложение (app.py)

```
from tkinter import *
from tkinter import ttk
import ctypes
from tkinter.messagebox import showinfo,showerror
import PythonDLL

def defineMethods():
    # region ОПРЕДЕЛЕНИЕ ФУНКЦИЙ
    Library.Insert.argtypes = [ctypes.c_int, ctypes.c_int, ctypes.c_int,
    ↪  ctypes.c_int]
    Library.Insert.restype = ctypes.c_int
    Library.Remove.argtypes = [ctypes.c_int, ctypes.c_int]
    Library.Remove.restype = ctypes.c_int
    Library.Clear.argtypes = []
    Library.Clear.restype = None
    Library.GetLength.argtypes = []
    Library.GetLength.restype = ctypes.c_int
    Library.FindByInf.argtypes = [ctypes.c_int, ctypes.POINTER(ctypes.c_int)]
    Library.FindByInf.restype = ctypes.POINTER(ctypes.c_int)
    Library.FindByChar.argtypes = [ctypes.c_int, ctypes.POINTER(ctypes.c_int)]
    Library.FindByChar.restype = ctypes.POINTER(ctypes.c_int)
    Library.FreeArray.argtypes = [ctypes.POINTER(ctypes.c_int)]
    Library.FreeArray.restype = None
    Library.GetInf.argtypes = [ctypes.c_int]
    Library.GetInf.restype = ctypes.c_int
    Library.GetChar.argtypes = [ctypes.c_int]
    Library.GetChar.restype = ctypes.c_int
    Library.GetPrev.argtypes = [ctypes.c_int]
```

```
28      Library.GetPrev.restype = ctypes.c_int
29      Library.GetNext.argtypes = [ctypes.c_int]
30      Library.GetNext.restype = ctypes.c_int
31      # endregion
32
33  libPascal=ctypes.CDLL("./PascalDLL.dll")
34  libCpp=ctypes.CDLL("./C++DLL.dll")
35  libC=ctypes.CDLL("./cDLL.dll")
36  pythonList=PythonDLL.List()
37  Library=libC  # по умолчанию C
38  defineMethods()
39  loaded_libraries={libPascal,libCpp,libC,pythonList}
40  LibIsPython=False
41
42  def cleanup():
43      for lib in loaded_libraries:
44          lib.Clear()
45          print(f"Библиотека: {lib} очищена")
46  def on_closing():
47      cleanup()
48      root.destroy()
49
50  #ОБРАБОТКА ВЫБОРА ДЕЙСТВИЯ (ДОБАВИТЬ/УДАЛИТЬ)
51  def comboBoxPL_selected(event):
52      global Library, LibIsPython, pythonList
53      LibIsPython=False
54      language=comboBoxPL.get()
55      if language == "Pascal":
56          Library = libPascal
57      elif language == "C++":
58          Library = libCpp
59      elif language == "C":
60          Library = libC
61      elif language=="Python":
62          '''if pythonList is None:
63              import PythonDLL
64              pythonList=PythonDLL.List()'''
65          LibIsPython=True
66          Library=pythonList
67      if not LibIsPython:
68          #loaded_libraries.add(Library)
69          defineMethods()
70      Update()
71  def item_selected(event):
72      selected=table.selection()
73      if selected:
74          entryIndex.delete(0,END)
75          entryIndex.insert(0,str(table.item(selected[0],'values')[0]))
76  def comboBoxAction_select(event):
77      groups={
78          "groupIndex":groupIndex,
79          "groupInf":groupInf,
80          "groupChar":groupChar,
81          "buttonAction":buttonAction #pady=(5,0)}
82      for group in groups.values():
83          group.pack_forget()
84      selected=comboBoxAction.get()
85      if "Удалить" in selected:
86          groupIndex.pack(fill="x")
87          buttonAction.pack(fill="x",pady=(5,0))
88      elif selected=="Добавить новый элемент":
89          groupInf.pack(fill="x")
90          groupChar.pack(fill="x")
91          buttonAction.pack(fill="x",pady=(5,0))
```

```
    else:
        groupIndex.pack(fill="x")
        groupInf.pack(fill="x")
        groupChar.pack(fill="x")
        buttonAction.pack(fill="x", pady=(5, 0))

def ClearList():
    for tlib in loaded_libraries:
        #Library.Clear()
        tlib.Clear()
    showinfo("","Список очищен")
    Update()

def GetLength():
    showinfo(title="Длина списка",message=f"Длина списка: {Library.GetLength()}")

def Update():
    for row in table.get_children():
        table.delete(row)
    n = Library.GetLength()
    for i in range(n):
        inf = Library.GetInf(i)
        ch_code = Library.GetChar(i)
        if ch_code:
            ch = chr(ch_code)
        else:
            ch=""
        next=Library.GetNext(i)
        prev=Library.GetPrev(i)
        table.insert("",END,values=(i+1,inf,ch,prev+1,next+1))

def Execute():
    res=0
    if "Добавить" in comboBoxAction.get():
        try:
            inf=int(entryInf.get())
        except:
            showerror("Ошибка", "Введите целое число")
            return
        ch_str=entryChar.get()
        if not(ch_str):
            showerror("Ошибка", "Введите символ")
            return
        if ch_str:
            ch = ord(ch_str[0])
            if not('a' <= entryChar.get()[0] <= 'z' or 'A' <= entryChar.get()[0] <= 'Z'):
                showerror("Ошибка", "Только латинский алфавит")
                return
        for tlib in loaded_libraries:
            if comboBoxAction.get()=="Добавить новый элемент":
                #res=Library.Insert(0, 0, inf, ch)
                res = tlib.Insert(0, 0, inf, ch)
            if comboBoxAction.get()=="Добавить элемент перед i-м":
                #res=Library.Insert(-1,int(entryIndex.get()),inf,ch)
                res = tlib.Insert(-1, int(entryIndex.get()), inf, ch)
            if comboBoxAction.get()=="Добавить элемент после i-го":
                #res=Library.Insert(1,int(entryIndex.get()),inf,ch)
                res = tlib.Insert(1, int(entryIndex.get()), inf, ch)
        entryInf.delete(0,END)
        entryChar.delete(0,END)
    if "Удалить" in comboBoxAction.get():
        for tlib in loaded_libraries:
```

```
154             if not entryIndex.get():
155                 return
156             if comboBoxAction.get()=="Удалить i-й элемент":
157                 #res=Library.Remove(0,int(entryIndex.get()))
158                 res = tlib.Remove(0, int(entryIndex.get()))
159             if "перед" in comboBoxAction.get():
160                 #res=Library.Remove(-1, int(entryIndex.get()))
161                 res = tlib.Remove(-1, int(entryIndex.get()))
162             if "после" in comboBoxAction.get():
163                 #res=Library.Remove(1, int(entryIndex.get()))
164                 res = tlib.Remove(1, int(entryIndex.get()))
165         entryIndex.delete(0,END)
166     if res!=0:
167         showerror("Ошибка", "Не удалось выполнить операцию")
168     Update()
169
170 def Find():
171     indices=[]
172     count=ctypes.c_int()
173     ptr=None
174     if mode.get()==1:
175         try:
176             inf=int(entryFind.get())
177         except:
178             showerror("Ошибка", "Введите целое число")
179             return
180         if not LibIsPython:
181             ptr=Library.FindByInf(inf,ctypes.byref(count))
182         else:
183             indices=Library.FindByInf(inf)
184     if mode.get()==2:
185         ch=entryFind.get()
186         if not(ch):
187             showerror("Ошибка", "Введите символ")
188             return
189         if not LibIsPython:
190             ptr = Library.FindByChar(ord(ch[0]), ctypes.byref(count))
191         else:
192             indices=Library.FindByChar(ch[0])
193     ucount=len(indices) if LibIsPython else count.value
194     if ucount==0:
195         showinfo("Поиск элементов", "Элементы не найдены")
196         return
197     if not LibIsPython and not ptr:
198         showerror("Ошибка", "Ошибка при поиске (ptr is null)")
199         return
200     try:
201         if not LibIsPython:
202             indices = [ptr[i] for i in range(ucount)]
203         showinfo("Результат поиска",f"Найдено элементов: {ucount}\n"f"Индексы:
            ↪ {', '.join(map(str, indices))}")
204     finally:
205         Library.FreeArray(0 if LibIsPython else ptr)
206
207 #region ИНТЕРФЕЙС
208 root=Tk()
209 root.title("Циклический двусвязный список (DLL)")
210 root.geometry("800x450")
211
212 #ВЫБОР ЯЗЫКА ПРОГРАММИРОВАНИЯ ДЛЯ ПОДКЛЮЧЕНИЯ ДИНАМИЧЕСКОЙ БИБЛИОТЕКИ
213 frameLeft=Frame()
214 framePL=Frame(frameLeft)
215 framePL.pack(anchor="center")
216 PL=["C","C++","Pascal","Python"]
```

```
labelPL=Label(framePL,text="Выбор ЯП для DLL")
labelPL.pack(fill="x")
comboBoxPL=ttk.Combobox(framePL,values=PL,width=25,state="readonly")
comboBoxPL.current(0)
comboBoxPL.pack(fill="x")
comboBoxPL.bind('<<ComboboxSelected>>', comboBoxPL_selected)

#frameLeft - ТАБЛИЦА ДЛЯ ЦИКЛИЧЕСКОГО ДВУСВЯЗНОГО СПИСКА
frameLeft.pack(side=LEFT,fill="both",expand=True)
frameCP=Frame(frameLeft) #CP - Command Panel
frameCP.pack(fill="x",pady=(5,10))
labelTable=Label(frameCP,text="Текущий список")
labelTable.pack(side=LEFT,expand=True, anchor="center")
buttonUpdate=Button(frameCP,text="Обновить список",command=Update)
buttonUpdate.pack(side=RIGHT,padx=17)
frameTable=Frame(frameLeft)
frameTable.pack(fill="both",expand=True)
scrollbar=ttk.Scrollbar(frameTable)
scrollbar.pack(side=RIGHT,fill="y")
columns=("index","number","char","last_prev","next_prev")
table=ttk.Treeview(frameTable,columns=columns,show="headings",
↪  yscrollcommand=scrollbar.set, height=15)
table.heading("index",text="Индекс")
table.heading("number",text="Число")
table.heading("char",text="Символ")
table.heading("last_prev",text="Номер предыдущего")
table.heading("next_prev",text="Номер следующего")
table.column("index", width=50, anchor="w")
table.column("number", width=100, anchor="w")
table.column("char", width=100, anchor="w")
table.column("last_prev", width=150, anchor="w")
table.column("next_prev", width=150, anchor="w")
table.pack(side=LEFT, expand=True,fill="both")
table.bind("<<TreeviewSelect>>",item_selected)
scrollbar.config(command=table.yview)

#frameRight - ФРЕЙМЫ ДЛЯ ДЕЙСТВИЙ СО СПИСКОМ
frameRight=Frame()
frameRight.pack(side=LEFT,fill="y",padx=5,pady=5)
frameAct=Frame(frameRight,borderwidth=3,relief="groove",padx=5,pady=5)
frameAct.pack(fill="x",pady=5)
buttonClear=Button(frameAct,text="Очистить список",command=ClearList)
buttonClear.pack(fill="x",pady=(0,5))
buttonLength=Button(frameAct,text="Определить длину списка",command=GetLength)
buttonLength.pack(fill="x")
frameChange=Frame(frameRight,borderwidth=3,relief="groove",padx=5,pady=5)
frameChange.pack(fill="x",pady=5)
labelChange=Label(frameChange,text="Изменение структуры")
labelChange.pack(fill="x")
actions=["Добавить новый элемент", "Добавить элемент перед i-м", "Добавить
↪  элемент после i-го",
         "Удалить i-й элемент", "Удалить элемент перед i-м", "Удалить элемент
         ↪  после i-го"]
comboBoxAction=ttk.Combobox(frameChange,values=actions,width=27,state="readonly")
comboBoxAction.current(0)
comboBoxAction.pack(fill="x")
comboBoxAction.bind('<<ComboboxSelected>>', comboBoxAction_select)
groupIndex=Frame(frameChange)
labelIndex=Label(groupIndex,text="Индекс")
labelIndex.pack(anchor="w")
entryIndex=Entry(groupIndex)
entryIndex.pack(fill="x")
groupInf=Frame(frameChange)
```

```
277 labelInf=Label(groupInf,text="Число")
278 labelInf.pack(anchor="w")
279 entryInf=Entry(groupInf)
280 entryInf.pack(fill="x")
281 groupInf.pack(fill="x")
282 groupChar=Frame(frameChange)
283 labelChar=Label(groupChar,text="Символ")
284 labelChar.pack(anchor="w")
285 entryChar=Entry(groupChar)
286 entryChar.pack(fill="x")
287 groupChar.pack(fill="x")
288 buttonAction=Button(frameChange,text="Выполнить",command=Execute)
289 buttonAction.pack(fill="x",pady=(5,0))
290
291 #frameFind ПОИСК ЗНАЧЕНИЙ В СПИСКЕ
292 frameFind=Frame(frameRight,borderwidth=3,relief="groove",padx=5,pady=5)
293 frameFind.pack(fill="x",pady=5)
294 labelFind=Label(frameFind,text="Поиск")
295 labelFind.pack()
296 mode = IntVar()
297 mode.set(1)
298 Radiobutton(frameFind, text="по числу", variable=mode,
    ↪  value=1).pack(anchor="w")
299 Radiobutton(frameFind, text="по символу", variable=mode,
    ↪  value=2).pack(anchor="w")
300 entryFind=Entry(frameFind)
301 entryFind.pack(fill="x")
302 buttonFind=Button(frameFind,text="Найти",command=Find)
303 buttonFind.pack(fill="x",pady=(5,0))
304 root.protocol("WM_DELETE_WINDOW", on_closing)
305 root.mainloop()
306 #endregion ИНТЕРФЕЙС
```

Критерий	**C**	**C++ STL**	**Pascal**	**Python**
Хранение	указатели вручную	готовый std::list	указатели вручную	объекты класса
Выделение памяти	malloc()	само	New()	само
Освобождение памяти	free()	само	Dispose()	само
Как экспортируются функции	Заголовочный файл	extern "C"+ __declspec (dllexport)	секция exports	методы класса
Как подключается в Python	ctypes.CDLL	ctypes.CDLL	ctypes.CDLL	import

Выводы

Было разработано графическое приложение с интерфейсом на Python при помощи стандартной библиотеки Tkinter. Приложение демонстрирует работу с циклическим двусвязным списком при помощи библиотек на четырёх языках программирования: C, C++, Pascal и Python.

Все четыре библиотеки реализуют одинаковый набор функций с единым интерфейсом, что позволяет переключаться между ними незаметно для пользователя.

Основная сложность заключалась в обеспечении совместимости модулей, написанных на разных языках. Каждый язык имеет свои особенности экспорта функций и управления памятью. Для C и Pascal используется явное выделение и освобождение памяти через `malloc`/`free` и `New`/`Dispose`, тогда как в C++ и Python управление памятью осуществляется автоматически.

Реализация на C++ отличается от остальных тем, что вместо ручного управления указателями используется стандартный контейнер STL `std::list`. Поскольку данный контейнер не является циклическим, цикличность списка было необходимо обеспечить при помощи остатка от деления: следующий элемент вычисляется как $(index + 1) \bmod length$, а предыдущий - как $(index - 1 + length) \bmod length$. Это позволяет сократить объём кода и упростить управление памятью.

Реализация на Python отличается тем, что не используются динамические библиотеки - модуль подключается напрямую через `import`. Вместо указателей используются ссылки на объекты классов, а память освобождается автоматически сборщиком мусора.

В ходе работы были освоены: создание динамических библиотек на разных языках программирования, подключение библиотек к Python через `ctypes`, разработка графического интерфейса с использованием Tkinter, обеспечение совместимости модулей с единым графическим интерфейсом, а также закреплено понимание работы циклического двусвязного списка через его визуальное представление.

Ссылка на репозиторий в GitHub:
https://github.com/umshik/Lab4.2